

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
Факультет физико-математических и естественных наук

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 9

Студент: Николаева Ангелина

Борисовна

Студенческий билет:

1032253612

Группа: НКАбд-04-25

МОСКВА

2026 г

Содержание

1. Цель работы	4
2. Выполнение лабораторной работы	5
2.1. Релаксация подпрограмм в NASM	5
2.1.1. Отладка программ с помощью GDB	8
2.1.2. Добавление точек останова	11
2.1.3. Работа с данными программы в GDB	12
2.1.4. Обработка аргументов командной строки в GDB	14
2.2. Задание для самостоятельной работы	15
3. Выводы	19

Список иллюстраций

1.	Создание рабочего каталога	5
2.	Запуск программы из листинга	6
3.	Изменение программы первого листинга	7
4.	Запуск программы в отладчике	9
5.	Проверка программы отладчиком	9
6.	Запуск отладчика с брейкпоинтом	10
7.	Дисассимилирование программы	10
8.	Режим псевдографики	11
9.	Просмотр установленных точек основы	11
10.	Просмотр содержимого регистров	12
11.	Просмотр содержимого переменных двумя способами	12
12.	Изменение содержимого переменных двумя способами	13
13.	Просмотр значения регистра разными представлениями	13
14.	Примеры использования команды set	14
15.	Подготовка новой программы	14
16.	Проверка работы стека	15
17.	Измененная программа предыдущей лабораторной работы	15
18.	Поиск ошибки в программе через пошаговую отладку	17
19.	Проверка корректировок в программе	18

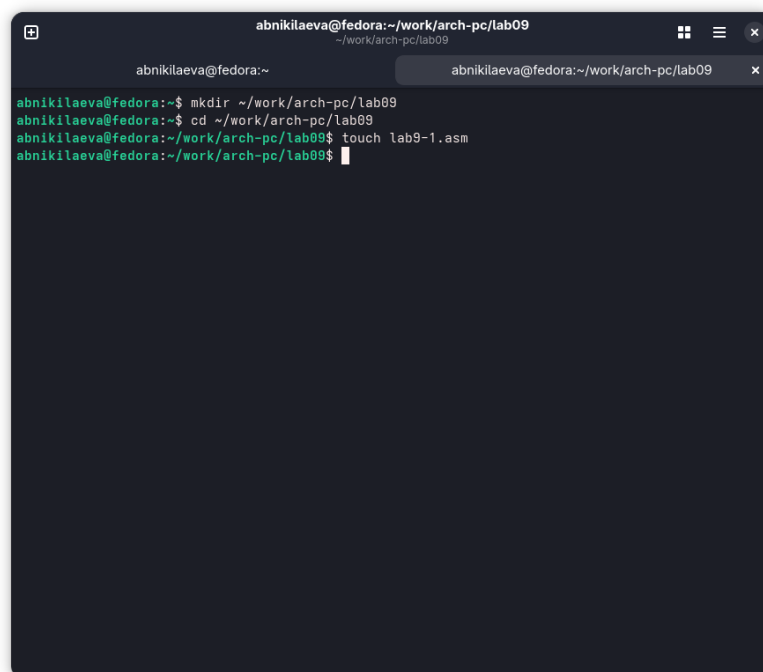
1. Цель работы

Приобретение навыков написания программ с использованием подпрограмм. Знакомство с методами отладки при помощи GDB и его основными возможностями.

2. Выполнение лабораторной работы

2.1. Релаксация подпрограмм в NASM

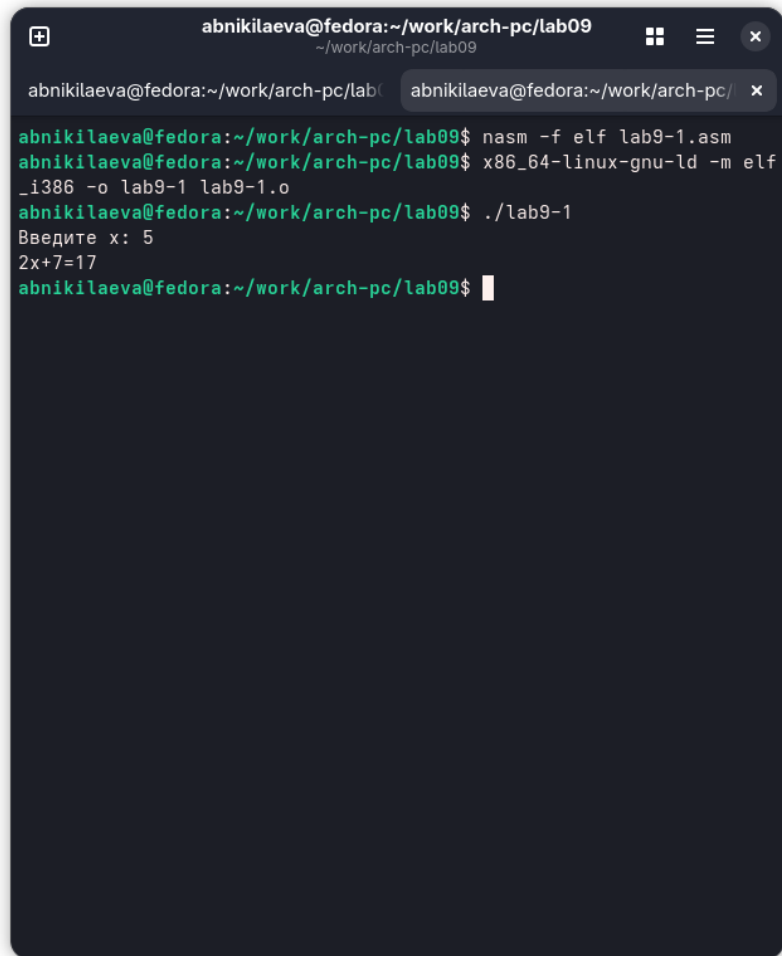
Создаю каталог для выполнения лабораторной работы №9 (рис. 4.1).

A screenshot of a terminal window with a dark background. The window title is 'abnikilaeva@fedora:~/work/arch-pc/lab09'. The terminal shows the following commands and output:

```
abnikilaeva@fedora:~$ mkdir ~/work/arch-pc/lab09
abnikilaeva@fedora:~$ cd ~/work/arch-pc/lab09
abnikilaeva@fedora:~/work/arch-pc/lab09$ touch lab9-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab09$
```

Рис. 4.1: Создание рабочего каталога

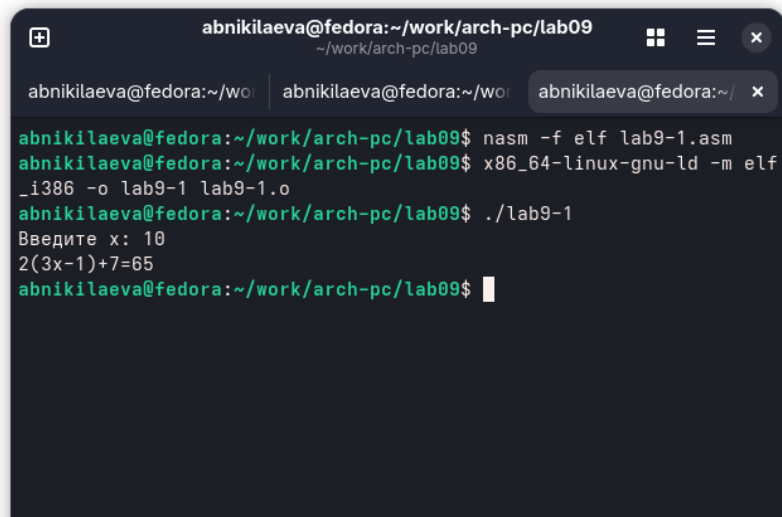
Копирую в файл код из листинга, компилирую и запускаю его, данная программа выполняет вычисление функции (рис.

A terminal window with a dark background and light green text. The window title is 'abnikilaeva@fedora:~/work/arch-pc/lab09'. The terminal shows the following commands and output:

```
abnikilaeva@fedora:~/work/arch-pc/lab09$ nasm -f elf lab9-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab09$ x86_64-linux-gnu-ld -m elf
_i386 -o lab9-1 lab9-1.o
abnikilaeva@fedora:~/work/arch-pc/lab09$ ./lab9-1
Введите x: 5
2x+7=17
abnikilaeva@fedora:~/work/arch-pc/lab09$
```

Рис. 4.2: Запуск программы из листинга

Изменяю текст программы, добавив в нее подпрограмму, теперь она вычисляет значение функции для выражения $f(g(x))$ (рис. 4.3).



```
abnikilaeva@fedora:~/work/arch-pc/lab09$ nasm -f elf lab9-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab09$ x86_64-linux-gnu-ld -m elf
_i386 -o lab9-1 lab9-1.o
abnikilaeva@fedora:~/work/arch-pc/lab09$ ./lab9-1
Введите x: 10
2(3x-1)+7=65
abnikilaeva@fedora:~/work/arch-pc/lab09$
```

Рис. 4.3: Изменение программы первого листинга

Код программы:

```
%include 'in_out.asm'

SECTION .data
msg: DB 'Введите x: ', 0
result: DB '2(3x-1)+7=', 0

SECTION .bss
x: RESB 80

res: RESB 80

SECTION .text
GLOBAL _start
_start:
mov eax, msg
call sprint

mov ecx, x
mov edx, 80
call sread

mov eax, x
```

```

call atoi

call _calcul

mov eax, result
call sprint
mov eax, [res]
call iprintLF
call quit

_calcul:
push eax
call _subcalcul

mov ebx, 2
mul ebx
add eax, 7

mov [res], eax
pop eax

ret

_subcalcul:
mov ebx, 3
mul ebx
sub eax, 1
ret

```

2.1.1.Отладка программ с помощью GDB

В созданный файл копирую программу второго листинга, транслирую с созданием файла листинга и отладки, komponую и запускаю в отладчике (рис. 4.4).


```
abnikilaeva@fedora:~/work/arch-pc/lab09$ nasm -f elf -g -l lab9-2.lst lab9-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab09$ x86_64-linux-gnu-ld -m elf_i386 -o lab9-2 lab9-2.o
abnikilaeva@fedora:~/work/arch-pc/lab09$ gdb lab9-2
bash: gdb: команда не найдена...
Аналогичная команда: 'gdb'
abnikilaeva@fedora:~/work/arch-pc/lab09$ gdb lab9-2
GNU gdb (Fedora Linux) 16.3-6.fc43
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "aarch64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-2...
(gdb)
```

Рис. 4.4: Запуск программы в отладчике

Запустив программу командой `run`, я убедился в том, что она работает исправно (рис. 4.5).

```
abnikilaeva@fedora:~/work/arch-pc/lab09 — gdb lab9-2
abnikilaeva@fedora:~/work/arch- abnikilaeva@fedora:~/work/arch- abnikilaeva@fedora:~/work/arch- abnikilaeva@fedora:~/work/arch-
GNU gdb (Fedora Linux) 16.3-6.fc43
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "aarch64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-2...
(gdb) run
Starting program: /home/abnikilaeva/work/arch-pc/lab09/lab9-2

This GDB supports auto-downloading debuginfo from the following URLs:
<linux=enforcing>
<https://debuginfod.fedoraproject.org/>
<linux=ignore>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Hello, world!
[Inferior 1 (process 11874) exited normally]
(gdb) break _start
Breakpoint 1 at 0x4000b0: file lab9-2.asm, line 16.
(gdb) run
Starting program: /home/abnikilaeva/work/arch-pc/lab09/lab9-2
```

Рис. 4.5: Проверка программы отладчиком

Для более подробного анализа программы добавляю брейкпоинт на метку `_start` и снова запускаю отладку (рис. 4.6).

```
abnikilaeva@fedora:~/work/arch-pc/lab09 — gdb lab9-2
abnikilaeva@fedora:~/work/arch: abnikilaeva@fedora:~/work/arch: abnikilaeva@fedora:~/work/arch: abnikilaeva@fedora:~/work/arch: abnikilaeva@fedora:~/work/a: x

There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "aarch64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-2...
(gdb) run
Starting program: /home/abnikilaeva/work/arch-pc/lab09/lab9-2

This GDB supports auto-downloading debuginfo from the following URLs:
<ima:enforcing>
<https://debuginfod.fedoraproject.org/>
<ima:ignore>
Enable debuginfo for this session? (y or [n]) y
Debuginfo has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Hello, world!
[Inferior 1 (process 11874) exited normally]
(gdb) break _start
Breakpoint 1 at 0x4000b0: file lab9-2.asm, line 16.
(gdb) run
Starting program: /home/abnikilaeva/work/arch-pc/lab09/lab9-2

Breakpoint 1, _start () at lab9-2.asm:16
16      mov x0, #1      // stdout
(gdb)
```

Рис. 4.6: Запуск отладчика с брейкпойнтом

Далее смотрю дисассимилированный код программы, перевожу на команд с синтаксисом Intel *amd* (рис. 4.7).

Различия между синтаксисом АТТ и Intel заключаются в порядке операндов (АТТ - Операнд источника указан первым. Intel - Операнд назначения указан первым), их размере (АТТ - размер операндов указывается явно с помощью суффиксов, непосредственные операнды предваряются символом \$; Intel - Размер операндов неявно определяется контекстом, как ah, eax, непосредственные операнды пишутся напрямую), именах регистров(АТТ - имена регистров предваряются символом %, Intel - имена регистров пишутся без префиксов).

```
abnikilaeva@fedora:~/work/arch-pc/lab09 — gdb lab9-2
abnikilaeva@fedora:~/work/arch: abnikilaeva@fedora:~/work/arch: abnikilaeva@fedora:~/work/arch: abnikilaeva@fedora:~/work/arch: abnikilaeva@fedora:~/work/a: x

=> 0x000000004000b0: <+0>: mov x0, #0x1      // #1
0x000000004000b4: <+4>: adr x1, 0x4100e4
0x000000004000b8: <+8>: mov x2, #0x7      // #7
0x000000004000bc: <+12>: mov x0, #0x40     // #64
0x000000004000c0: <+16>: svc #0x0
0x000000004000c4: <+20>: mov x0, #0x1      // #1
0x000000004000c8: <+24>: adr x1, 0x4100ec
0x000000004000cc: <+28>: mov x2, #0x7      // #7
0x000000004000d0: <+32>: mov x8, #0x40     // #64
0x000000004000d4: <+36>: svc #0x0
0x000000004000d8: <+40>: mov x0, #0x0      // #0
0x000000004000dc: <+44>: mov x8, #0x5d     // #93
0x000000004000e0: <+48>: svc #0x0

End of assembler dump.
(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x000000004000b0: <+0>: mov x0, #0x1      // #1
0x000000004000b4: <+4>: adr x1, 0x4100e4
0x000000004000b8: <+8>: mov x2, #0x7      // #7
0x000000004000bc: <+12>: mov x0, #0x40     // #64
0x000000004000c0: <+16>: svc #0x0
0x000000004000c4: <+20>: mov x0, #0x1      // #1
0x000000004000c8: <+24>: adr x1, 0x4100ec
0x000000004000cc: <+28>: mov x2, #0x7      // #7
0x000000004000d0: <+32>: mov x8, #0x40     // #64
0x000000004000d4: <+36>: svc #0x0
0x000000004000d8: <+40>: mov x0, #0x0      // #0
0x000000004000dc: <+44>: mov x8, #0x5d     // #93
0x000000004000e0: <+48>: svc #0x0

End of assembler dump.
(gdb) |
```

Рис. 4.7: Дисассимилирование программы

Включаю режим псевдографики для более удобного анализа программы (рис. 4.8).

```

Register group: general
eax 0x0 0 ecx 0x0 0
edx 0x0 0 ebx 0x0 0
esp 0xffffcf10 0xffffcf10 ebp 0x0 0
esi 0x0 0 edi 0x0 0
eip 0x8049000 0x8049000 <_start> eflags 0x202 [ IF ]
cs 0x23 35 ss 0x2b 43
ds 0x2b 43 es 0x2b 43
fs 0x0 0 gs 0x0 0

0x8049000 <_start> mov eax,0x4
0x8049005 <_start+5> mov ebx,ecx
0x804900a <_start+10> mov ecx,0x8049000
0x804900f <_start+15> mov edx,ecx
0x8049014 <_start+20> int $eax
0x8049016 <_start+22> mov eax,ecx
0x804901b <_start+27> mov ebx,ecx
0x8049020 <_start+32> mov ecx,0x8049000
0x8049025 <_start+37> mov edx,ecx
0x804902a <_start+42> int $eax

native process 5896 (asm) In: _start L11 PC: 0x8049000
(gdb) layout regs
(gdb)

```

Рис. 4.8: Режим псевдографики

2.1.2.Добавление точек останова

С помощью команды info breakpoints проверяю установленные точки останова, устанавливаю еще одну точку останова по для предпоследней инструкции(рис.4.9)

```

lab9-2.asm
14 _start:
15 // write syscall
16 mov x0, #1 // stdout
17 adr x1, msg1 // buffer
18 mov x2, #7 // length of "Hello, "
19 mov x8, #64 // write syscall number

B>0x4000b0 <_start> mov x0,#0x1 // #1
0x4000b4 <_start+4> adr x1,0x4100e4
0x4000b8 <_start+8> mov x2,#0x7 // #7
0x4000bc <_start+12> mov x0,#0x40 // #64
0x4000c0 <_start+16> svc #0x0
0x4000c4 <_start+20> mov x0,#0x1 // #1
0x4000c8 <_start+24> adr x1,0x4100ec

native process 11882 (asm) In: _start L16 PC: 0x4000b0
(gdb) layout split
(gdb) break *0x4000dc
Breakpoint 2 at 0x4000dc: file lab9-2.asm, line 30.
(gdb) i b
Num Type Disp Enb Address What
1 breakpoint keep y 0x000000004000b0 lab9-2.asm:16
2 breakpoint already hit 1 time
(gdb)

```

Рис. 4.9: Просмотр установленных точек основы

2.1.3. Работа с данными программы в GDB

Просматриваю содержимое регистров командой `info registers` (рис. 4.10).



The screenshot shows a GDB session with the `info registers` command. The output lists registers x0 through x7, all containing 0x0. A red box highlights the register list. The assembly code for the `_start` function is visible in the background, showing instructions like `mov x0, #0` and `svc #0`.

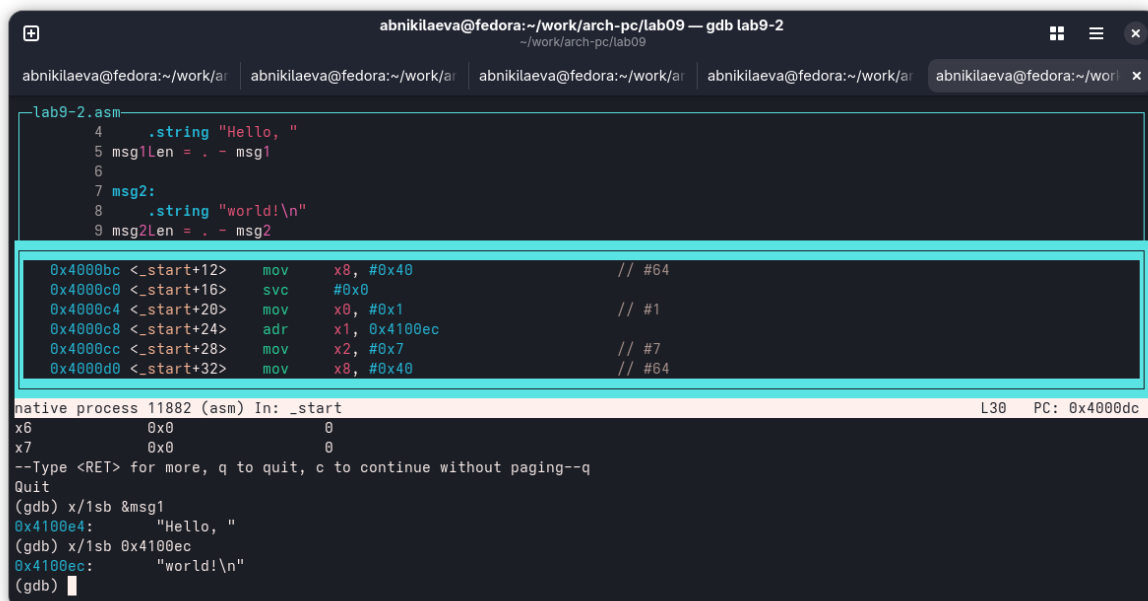
```
abnikilaeva@fedora:~/work/arch-pc/lab09 — gdb lab9-2
abnikilaeva@fedora:~/work/arch-pc/lab09
14 _start:
28 // exit syscall
B> 29 mov x0, #0 // stdout
30 adr x0, #93 // exit syscall number
31 svc #0 of "Hello, "

0x400134 udf #2
0x4000d0 <_start+32> mov x8, #0x40 // #64
0x4000d4 <_start+36> svc #0x0
0x4000d8 <_start+40> mov x0, #0x0 // #0
B> 0x4000dc <_start+44> mov x8, #0x5d // #93
0x4000e0 <_start+48> svc #0x0 undefined
0x4000e4 ldnp d8, d25, [x10, #-320]
0x4000e8 .inst 0x00202c6f ; NYI

native process 11882 (asm) in: _start L16 PC: 0x4000b0
Breakpoint 2 at 0x4000dc: file lab9-2.asm, line 30.
30
x0 0x0 0
x1 0x4100ec 4260076
x2 0x7 7
x3db co 0x0 0
x4 0x0 0
x5 0x0 0
x6 0x0 0
x7 0x0 0
--Type <RET> for more, q to quit, c to continue without paging--
```

Рис. 4.10: Просмотр содержимого регистров

Смотрю содержимое переменных по имени и по адресу (рис. 4.11).



The screenshot shows a GDB session with the `info registers` command. The output lists registers x0 through x7, all containing 0x0. A red box highlights the register list. The assembly code for the `_start` function is visible in the background, showing instructions like `mov x0, #0` and `svc #0`.

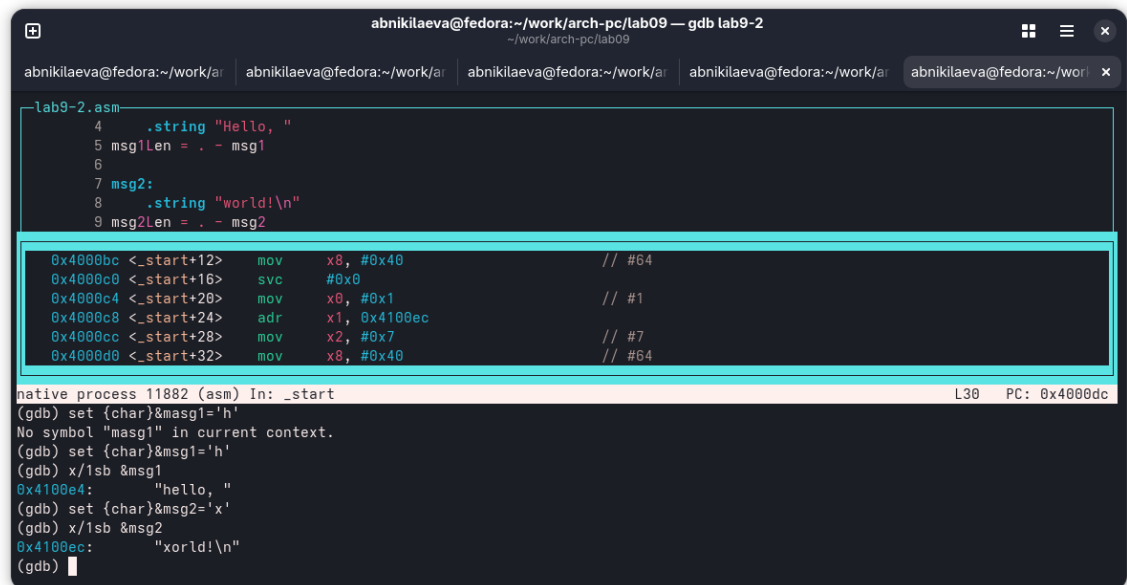
```
abnikilaeva@fedora:~/work/arch-pc/lab09 — gdb lab9-2
abnikilaeva@fedora:~/work/arch-pc/lab09
lab9-2.asm
4 .string "Hello, "
5 msg1Len = . - msg1
6
7 msg2:
8 .string "world!\n"
9 msg2Len = . - msg2

0x4000bc <_start+12> mov x8, #0x40 // #64
0x4000c0 <_start+16> svc #0x0
0x4000c4 <_start+20> mov x0, #0x1 // #1
0x4000c8 <_start+24> adr x1, 0x4100ec
0x4000cc <_start+28> mov x2, #0x7 // #7
0x4000d0 <_start+32> mov x8, #0x40 // #64

native process 11882 (asm) in: _start L30 PC: 0x4000dc
x6 0x0 0
x7 0x0 0
--Type <RET> for more, q to quit, c to continue without paging--
Quit
(gdb) x/1sb &msg1
0x4100e4: "Hello, "
(gdb) x/1sb 0x4100ec
0x4100ec: "world!\n"
(gdb)
```

Рис. 4.11: Просмотр содержимого переменных двумя способами

Меняю содержимое переменных по имени и по адресу (рис. 4.12).



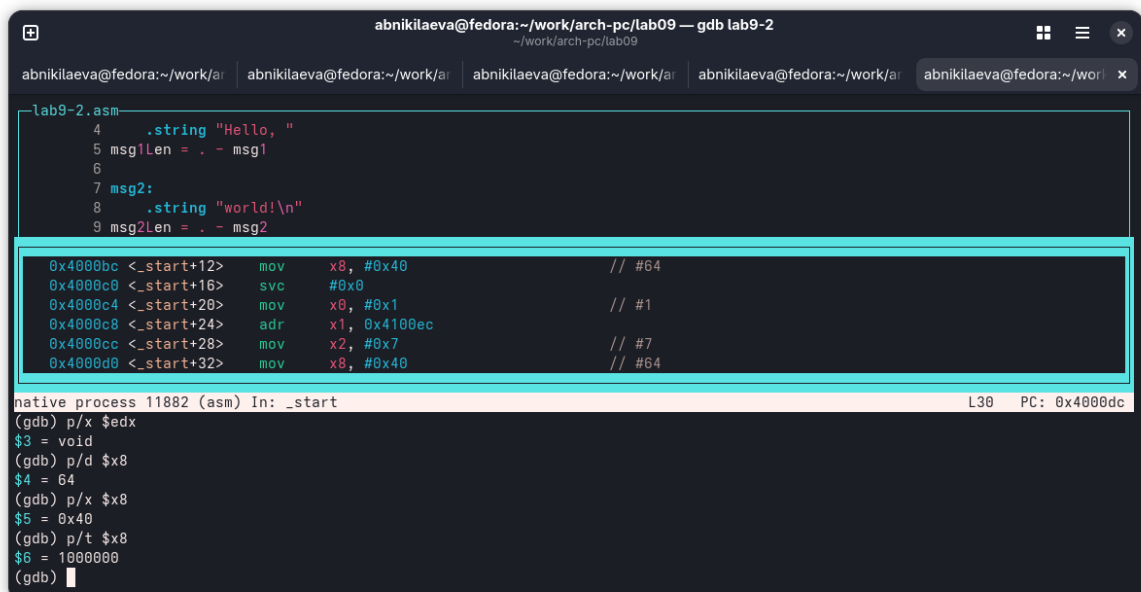
```
abnikilaeva@fedora:~/work/arch-pc/lab09 — gdb lab9-2
abnikilaeva@fedora:~/work/arch-pc/lab09
lab9-2.asm
4      .string "Hello, "
5      msg1Len = . - msg1
6
7      msg2:
8      .string "world!\n"
9      msg2Len = . - msg2

0x4000bc <_start+12>  mov     x8, #0x40          // #64
0x4000c0 <_start+16>  svc     #0x0
0x4000c4 <_start+20>  mov     x0, #0x1          // #1
0x4000c8 <_start+24>  adr     x1, 0x4100ec
0x4000cc <_start+28>  mov     x2, #0x7          // #7
0x4000d0 <_start+32>  mov     x8, #0x40          // #64

native process 11882 (asm) In: _start L30 PC: 0x4000dc
(gdb) set (char)&msg1='h'
No symbol "msg1" in current context.
(gdb) set (char)&msg1='h'
(gdb) x/1sb &msg1
0x4100e4: "hello, "
(gdb) set (char)&msg2='x'
(gdb) x/1sb &msg2
0x4100ec: "xorld!\n"
(gdb) 
```

Рис. 4.12: Изменение содержимого переменных двумя способами

Вывожу в различных форматах значение регистра edx (рис. 13)



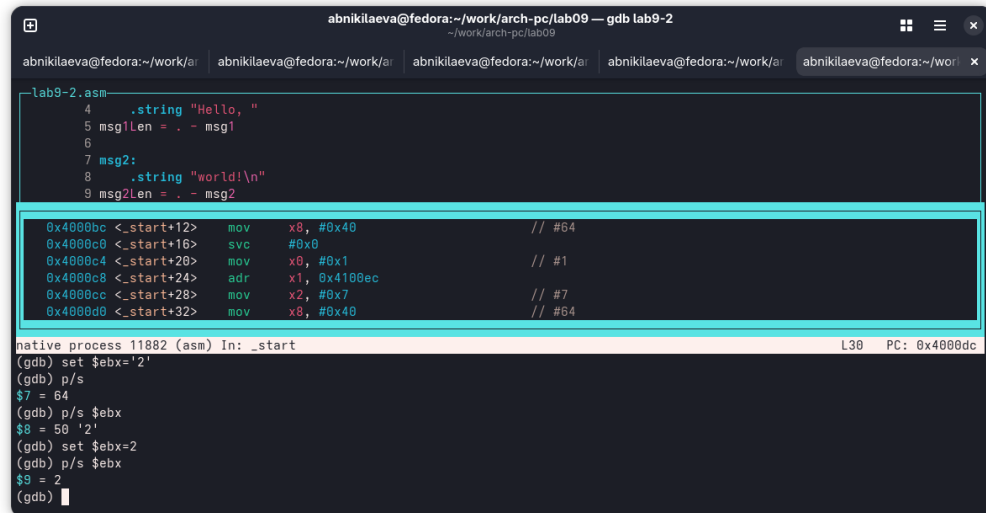
```
abnikilaeva@fedora:~/work/arch-pc/lab09 — gdb lab9-2
abnikilaeva@fedora:~/work/arch-pc/lab09
lab9-2.asm
4      .string "Hello, "
5      msg1Len = . - msg1
6
7      msg2:
8      .string "world!\n"
9      msg2Len = . - msg2

0x4000bc <_start+12>  mov     x8, #0x40          // #64
0x4000c0 <_start+16>  svc     #0x0
0x4000c4 <_start+20>  mov     x0, #0x1          // #1
0x4000c8 <_start+24>  adr     x1, 0x4100ec
0x4000cc <_start+28>  mov     x2, #0x7          // #7
0x4000d0 <_start+32>  mov     x8, #0x40          // #64

native process 11882 (asm) In: _start L30 PC: 0x4000dc
(gdb) p/x $edx
$3 = void
(gdb) p/d $x8
$4 = 64
(gdb) p/x $x8
$5 = 0x40
(gdb) p/t $x8
$6 = 1000000
(gdb) 
```

Рис. 4.13: Просмотр значения регистра разными представлениями

С помощью команды set меняю содержимое регистра ebx (рис. 4.14).



The screenshot shows a GDB terminal window with the title 'abnikilaeva@fedora:~/work/arch-pc/lab09 — gdb lab9-2'. The main pane displays assembly code for 'lab9-2.asm'. A cyan box highlights a block of assembly instructions:
0x4000bc <_start+12> mov x8, #0x40 // #64
0x4000c0 <_start+16> svc #0x0
0x4000c4 <_start+20> mov x0, #0x1 // #1
0x4000c8 <_start+24> adr x1, 0x4100ec
0x4000cc <_start+28> mov x2, #0x7 // #7
0x4000d0 <_start+32> mov x8, #0x40 // #64
The bottom pane shows the GDB prompt and several 'set' commands:
(gdb) set \$ebx='2'
(gdb) p/s
\$7 = 64
(gdb) p/s \$ebx
\$8 = 50 '2'
(gdb) set \$ebx=2
(gdb) p/s \$ebx
\$9 = 2
(gdb)

Рис. 4.14: Примеры использования команды set

2.1.4. Обработка аргументов командной строки в GDB

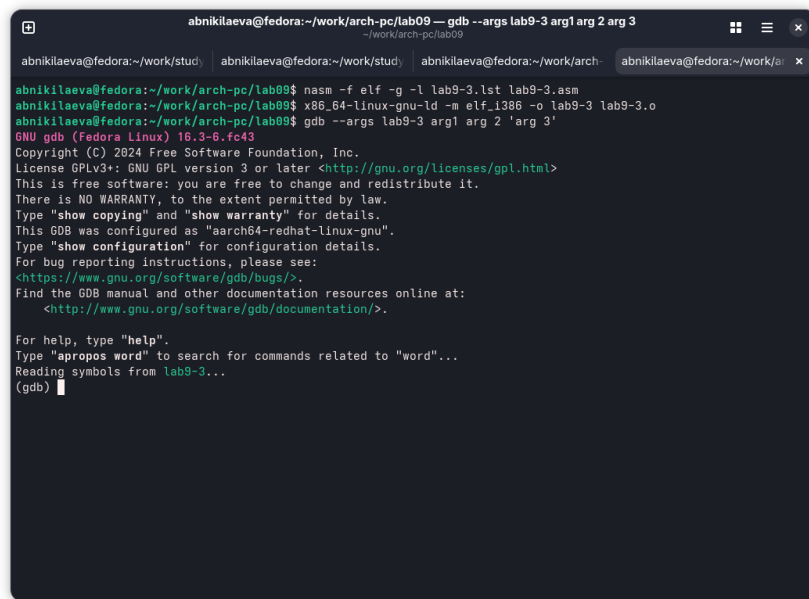
Копирую программу из предыдущей лабораторной работы в текущий каталог и и создаю исполняемый файл с файлом листинга и отладки (рис. 4.15).



The screenshot shows a terminal window with the title 'abnikilaeva@fedora:~/work/arch-pc/lab09'. The terminal displays the following commands and output:
abnikilaeva@fedora:~/work/arch-pc/lab09\$ cp ~/work/arch-pc/lab08/lab8-2.asm ~/work/arch-pc/lab09/lab9-3.asm
abnikilaeva@fedora:~/work/arch-pc/lab09\$ nasm -f elf -g -l lab9-3.lst lab9-3.asm
abnikilaeva@fedora:~/work/arch-pc/lab09\$ x86_64-linux-gnu-ld -m elf_i386 -o lab9-3 lab9-3.o
abnikilaeva@fedora:~/work/arch-pc/lab09\$

Рис. 4.15: Подготовка новой программы

Программа lab9-3 была создана. При попытке запуска возникла ошибка из-за несовместимости архитектур (aarch64 и i386).



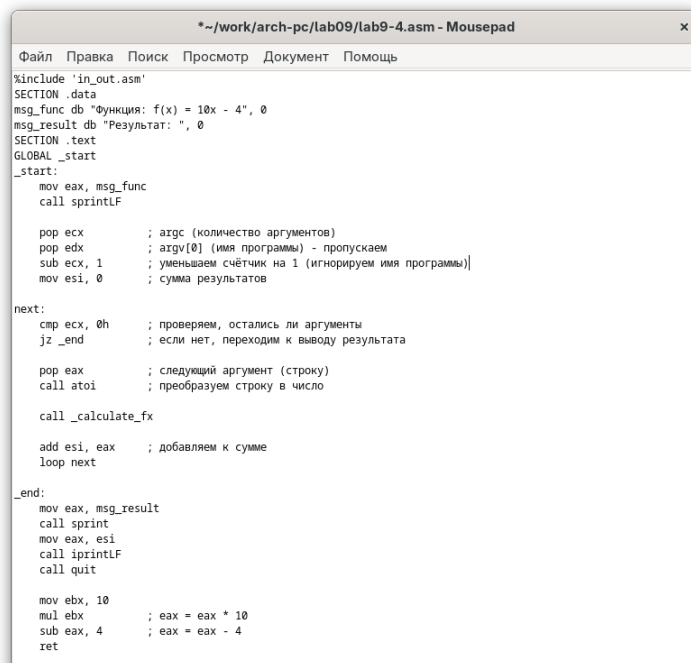
```
abnikilaeva@fedora:~/work/arch-pc/lab09 — gdb --args lab9-3 arg1 arg2 arg3
~/work/arch-pc/lab09
abnikilaeva@fedora:~/work/study | abnikilaeva@fedora:~/work/study | abnikilaeva@fedora:~/work/arch- | abnikilaeva@fedora:~/work/arch-
abnikilaeva@fedora:~/work/arch-pc/lab09$ nasm -f elf -g -l lab9-3.lst lab9-3.asm
abnikilaeva@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-3 lab9-3.o
abnikilaeva@fedora:~/work/arch-pc/lab09$ gdb --args lab9-3 arg1 arg2 arg3
GNU gdb (Fedora Linux) 16.3-6.fc43
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "aarch64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-3...
(gdb)
```

Рис. 4.16

2.2.Задание для самостоятельной работы

1. Меняю программу самостоятельной части предыдущей лабораторной работы с использованием подпрограммы (рис. 4.17).



```
*~/work/arch-pc/lab09/lab9-4.asm - Mousepad
Файл  Правка  Поиск  Просмотр  Документ  Помощь

%include 'in_out.asm'
SECTION .data
msg_func db "Функция: f(x) = 10x - 4", 0
msg_result db "Результат: ", 0
SECTION .text
GLOBAL _start
_start:
    mov eax, msg_func
    call sprintf

    pop ecx        ; args (количество аргументов)
    pop edx        ; argv[0] (имя программы) - пропускаем
    sub ecx, 1     ; уменьшаем счётчик на 1 (игнорируем имя программы)
    mov esi, 0     ; сумма результатов

next:
    cmp ecx, 0h    ; проверяем, остались ли аргументы
    jz _end        ; если нет, переходим к выводу результата

    pop eax        ; следующий аргумент (строку)
    call atoi      ; преобразуем строку в число

    call _calculate_fx

    add esi, eax   ; добавляем к сумме
    loop next

_end:
    mov eax, msg_result
    call sprintf
    mov eax, esi
    call iprintf
    call quit

    mov ebx, 10
    mul ebx        ; eax = eax * 10
    sub eax, 4     ; eax = eax - 4
    ret
```

Рис. 4.17: Измененная программа предыдущей лабораторной работы

Код программы:

```
%include 'in_out.asm'

SECTION .data
msg_func db "Функция: f(x) = 10x - 4", 0
msg_result db "Результат: ", 0

SECTION .text
GLOBAL _start

_start:
mov eax, msg_func
call sprintf

pop ecx
pop edx

sub ecx, 1
mov esi, 0

next:
cmp ecx, 0h
jz _end
pop eax
call atoi

call _calculate_fx

add esi, eax
loop next

_end:
mov eax, msg_result
call sprintf
mov eax, esi
```



```
call iprintLF
call quit
```

```
_calculate_fx:
```

```
mov ebx, 10
```

```
mul ebx
```

```
sub eax, 4
```

2. Запускаю программу в режиме отладчика и пошагово через si просматриваю изменение значений регистров через i r. При выполнении инструкции mul esx можно заметить, что результат умножения записывается в регистр eax, но также меняет и edx. Значение регистра ebx не обновляется напрямую, поэтому результат программа неверно подсчитывает функцию (рис. 4.18).

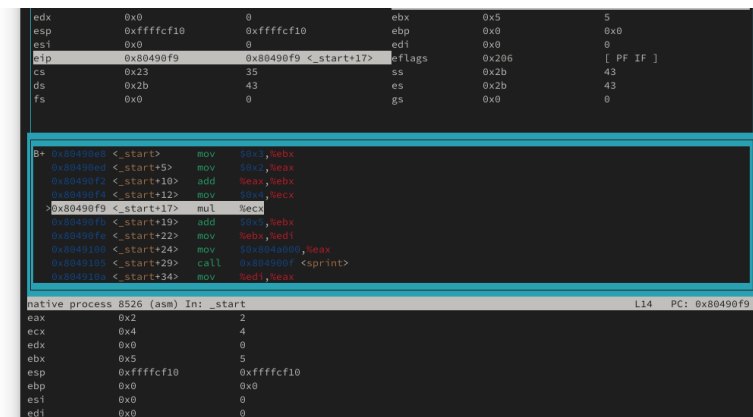


Рис. 4.18: Поиск ошибки в программе через пошаговую отладку

Исправляю найденную ошибку, теперь программа верно считает значение функции (рис. 4.19).

```
abnikilaeva@fedora:~/work/arch-pc/lab09
~/work/arch-pc/lab09

abnikilaeva@fedora:~/work/arch-pc/lab09$ touch lab9-5.asm
abnikilaeva@fedora:~/work/arch-pc/lab09$ mousepad lab9-5.asm
abnikilaeva@fedora:~/work/arch-pc/lab09$ nasm -f elf lab9-5.asm
abnikilaeva@fedora:~/work/arch-pc/lab09$ x86_64-linux-gnu-ld -m elf_i386 -o lab9-5 lab9-5.o
abnikilaeva@fedora:~/work/arch-pc/lab09$ mousepad lab9-5.asm
abnikilaeva@fedora:~/work/arch-pc/lab09$ nasm -f elf lab9-5.asm
abnikilaeva@fedora:~/work/arch-pc/lab09$ x86_64-linux-gnu-ld -m elf_i386 -o lab9-5 lab9-5.o
abnikilaeva@fedora:~/work/arch-pc/lab09$ ./lab9-5

abnikilaeva@fedora:~/work/arch-pc/lab09$
```

Рис. 4.19: Проверка корректировок в программе

Код измененной программы:

```
%include 'in_out.asm'

SECTION .data
div: DB 'Результат: ', 0

SECTION .text
GLOBAL _start

_start:
mov ebx, 3
mov eax, 2

add ebx, eax

mov eax, ebx

mov ecx, 4

mul ecx

add eax, 5

mov edi, eax

mov eax, div
call sprint

mov eax, edi
call iprintLF

call quit
```

3. Выводы

В результате выполнения данной лабораторной работы я приобрел навыки написания программ с использованием подпрограмм, а так же познакомился с методами отладки при помощи GDB и его основными возможностями.